

UNIT 2 · TECHNOLOGY FOR IoT

Different Frameworks for IoT

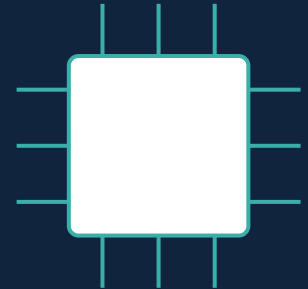
Firmware & device-side frameworks: Arduino, ESP-IDF, FreeRTOS, and MicroPython

Arduino Framework

ESP-IDF

FreeRTOS Tasks

MicroPython



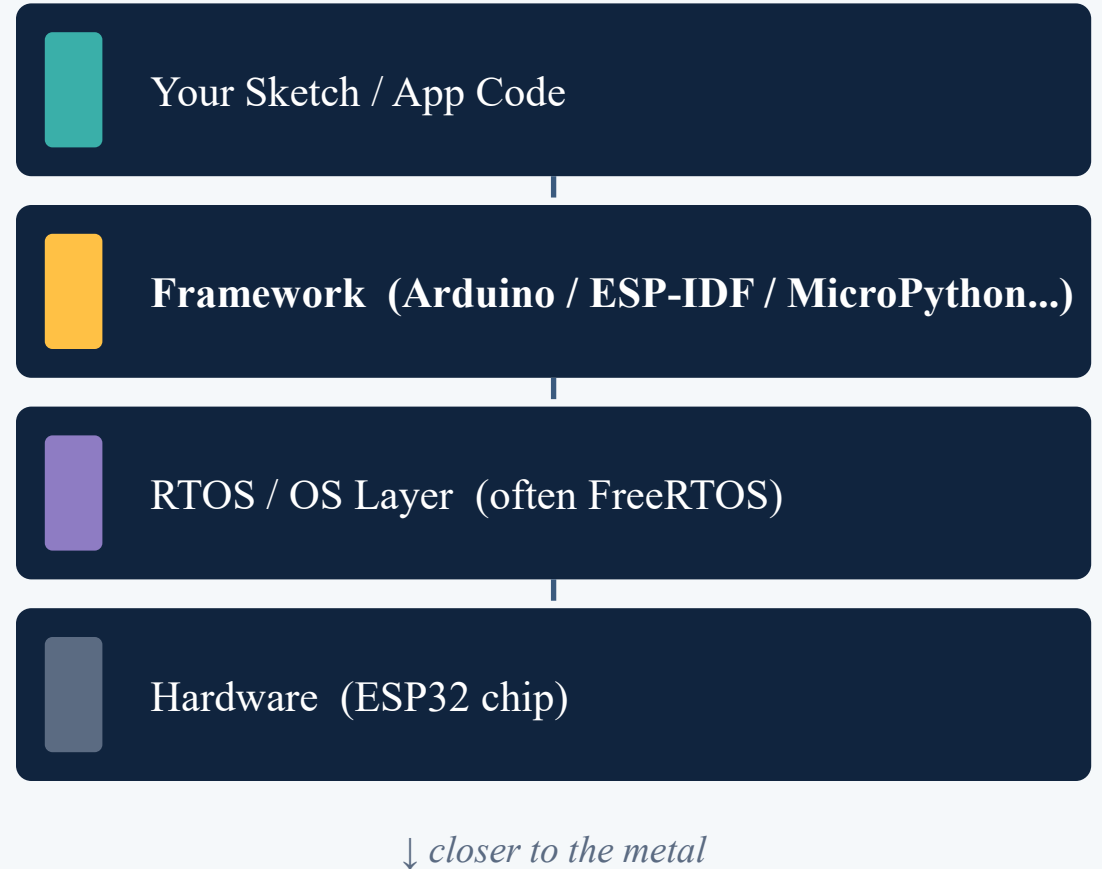
What We'll Cover Today

- 01 What is a "framework"?
- 02 Recap: the Arduino framework
- 03 ESP-IDF — the native framework
- 04 RTOS concept & FreeRTOS tasks
- 05 MicroPython / CircuitPython
- 06 Comparison & choosing the right one

What Is a "Framework," Exactly?

- A framework **is the software layer between your code and the raw hardware.**
- It **gives you ready-made functions** (pinMode, digitalWrite...) instead of making you configure registers by hand.
- You've been using one this whole time — the Arduino framework.
- Different frameworks make different trade-offs:

ease of use vs **control** vs **performance** vs **how many things can run "at once."**



Recap: The Arduino Framework

- A C/C++ abstraction layer + IDE + massive library ecosystem.
- On the ESP32, the Arduino framework is actually built on top of ESP-IDF underneath.
- Structured around two functions: `setup()` (runs once) and `loop()` (runs forever).
- This is exactly what you've used in every practical so far.



ARDUINO FRAMEWORK

```
void setup() {  
    pinMode(ledPin, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(ledPin, HIGH);  
    delay(1000);  
    digitalWrite(ledPin, LOW);  
    delay(1000);  
}
```

✓ **Strengths:** huge community, tons of libraries, beginner-friendly, fastest way to get something working.

...But Arduino Has a Catch

- `loop()` runs everything sequentially, one line after another — nothing else happens while a line is executing.
- `delay()` blocks the entire program. While it waits, the board can't read a sensor, respond to a button, or do anything else.
- Fine for a simple blink demo. **A real problem** once a project needs multiple things happening "at the same time."

THE PROBLEM

`delay(5000)` freezes EVERYTHING for 5 full seconds — sensors, buttons, WiFi handling — all paused.

THE FIX WE'LL SEE TODAY

Two ways out: write closer to the hardware with ESP-IDF, or restructure code into independent FreeRTOS tasks that don't block each other.

ESP-IDF — The Native Framework

- IDF = IoT Development Framework — Espressif's own official framework for the ESP32.
- Written in plain C, and the Arduino-ESP32 core is actually built on top of it.
- **Gives direct access to low-level features Arduino hides:** fine-grained WiFi config, deep-sleep tuning, precise timer/interrupt control.
- Built on FreeRTOS underneath — **multitasking is available natively.**

ARDUINO IS BUILT ON ESP-IDF

- Arduino Framework
- ESP-IDF
- FreeRTOS + Hardware

Trade-off: steeper learning curve, more boilerplate code — you write more to do the same simple task, in exchange for more control.

Same Task, Two Frameworks

Blinking an LED — Arduino framework vs raw ESP-IDF

ARDUINO

```
void setup() {  
    pinMode(2, OUTPUT);  
}  
void loop() {  
    digitalWrite(2, HIGH);  
    delay(1000);  
    digitalWrite(2, LOW);  
    delay(1000);  
}
```

ESP-IDF

```
void app_main(void) {  
    gpio_set_direction(2, GPIO_MODE_OUTPUT);  
    while (1) {  
        gpio_set_level(2, 1);  
        vTaskDelay(1000 / portTICK_PERIOD_MS);  
        gpio_set_level(2, 0);  
        vTaskDelay(1000 / portTICK_PERIOD_MS);  
    }  
}
```

Mapping the Concepts Across

`setup() + loop()` → `app_main()` with a `while(1)` loop inside

`pinMode()` → `gpio_set_direction()`

`digitalWrite()` → `gpio_set_level()`

`delay(ms)` → `vTaskDelay(ms / portTICK_PERIOD_MS)`

Same underlying idea, different level of abstraction — everything Arduino does, ESP-IDF does more explicitly.

Why Do We Need an RTOS?

- Real IoT **projects often need several things happening independently**: blink an LED, read a sensor, check WiFi, respond to a button — all "at once."
- A single loop() with delay() calls can't do this — one blocking call freezes everything else.
- An RTOS (Real-Time Operating System) lets you split code into independent tasks that the chip switches between rapidly.
- The **ESP32 has FreeRTOS built in** — even Arduino sketches secretly run inside one FreeRTOS task already.

ONE loop() — SEQUENTIAL

Blink LED → wait → read sensor
→ wait → check WiFi → ...

MULTIPLE TASKS — CONCURRENT

- Task 1: Blink LED
- Task 2: Read sensor
- Task 3: Handle WiFi

FreeRTOS Tasks — In Code

- **xTaskCreate()**

spins up a new independent task — its own function, running alongside everything else.

- **Inside a task, vTaskDelay()**

pauses only that task — other tasks keep running.

- You can create a task from setup() itself, even in the Arduino framework, using its underlying FreeRTOS support.
- loop() can end up doing nothing at all — the real work happens in tasks.

```
ARDUINO + FreeRTOS

void blinkTask(void *pvParameters) {
    pinMode(2, OUTPUT);
    while (1) {
        digitalWrite(2, HIGH);
        vTaskDelay(500 / portTICK_PERIOD_MS);
        digitalWrite(2, LOW);
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
}

void setup() {
    xTaskCreate(blinkTask, "Blink", 1000, NULL,
1, NULL);
}

void loop() {
    // empty -- task handles blinking
}
```

FreeRTOS Tasks — In Code

- **xTaskCreate()**

```
xTaskCreate(blinkTask, "Blink", 1000, NULL, 1, NULL);
```

#	Argument	Example Value	Meaning
1	Function	blinkTask	Which function becomes the new task
2	Name	"Blink"	Just a label, useful for debugging (shows up in task lists)
3	Stack size	1000	How many bytes of stack this task gets — separate from the main loop() task's stack
4	Parameters	NULL	Nothing passed into blinkTask's pvParameters (used to pass data into the task at creation)
5	Priority	1	Scheduling priority — higher number = runs more eagerly when tasks compete for CPU time
6	Task handle	NULL	Not keeping a reference to control/delete this task later

delay() vs vTaskDelay() — The Key Difference

delay()

- Freezes the ENTIRE program.
- Nothing else can run — no other task, no sensor check, nothing.
- Fine only when nothing else needs to happen during the wait.

vTaskDelay()

- Pauses only the CURRENT task.
- Other tasks keep running normally during that time.
- This is what makes true multitasking possible on the ESP32.

MicroPython / CircuitPython

- A completely different approach: write and run actual Python directly on the microcontroller.
- No separate compile step — flash MicroPython once, then just copy/edit .py files onto the board.
- Comes with a REPL — an interactive prompt where you can type a command and see it run instantly on the hardware.
- CircuitPython (by Adafruit) is a friendly variant, popular in education and beginner boards.

WHY IT FEELS DIFFERENT

- **No compiling** — edit → save → it just runs
- **Interactive REPL** — type a line, see it execute live
- **Python syntax** — no semicolons, no curly braces

Same Blink, in MicroPython

ARDUINO (C++)

```
void setup() {  
  pinMode(2, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(2, HIGH);  
  delay(1000);  
  digitalWrite(2, LOW);  
  delay(1000);  
}
```

MICROPYTHON

```
from machine import Pin  
import time  
  
led = Pin(2, Pin.OUT)  
  
while True:  
    led.value(1)  
    time.sleep(1)  
    led.value(0)  
    time.sleep(1)
```

MicroPython — Trade-offs

✓ ADVANTAGES

- Much faster to prototype — no compile/upload cycle.
- Interactive REPL for quick testing and debugging.
- Gentler syntax — great for teaching programming concepts.

✗ LIMITATIONS

- Slower execution than compiled C/C++.
- Higher RAM overhead — less headroom for large projects.
- Not ideal for tight timing or high-performance tasks.

Framework Comparison

FRAMEWORK	BEST FOR	LEARNING CURVE	MULTITASKING
Arduino	Quick prototypes, learning, huge library support	Low	Manual only
ESP-IDF	Production firmware, fine-grained hardware control	High	Native (FreeRTOS)
FreeRTOS tasks	Multiple concurrent operations in real products	Medium–High	Native
MicroPython	Rapid scripting, education, quick experiments	Low	Limited

Which One Should I Actually Use?

Just starting out / a class project?

Arduino framework

Need max performance or deep hardware access?

ESP-IDF

Project needs several things running independently?

FreeRTOS tasks

Want to prototype fast, or learning programming basics?

MicroPython

What You Should Take Away

Framework = the abstraction layer between code and hardware — you've always used one.

Arduino = easiest to start, but single-threaded and `delay()` blocks everything.

ESP-IDF = native, more control, more code — what Arduino is built on.

FreeRTOS tasks = the real fix for "multiple things at once" — `vTaskDelay()` doesn't block other tasks.

MicroPython = fastest to prototype, but slower and less memory-efficient than compiled C.